

데이터 분석, 모델 vs 전처리

* 모델링(Modeling) : 데이터의 패턴을 학습해 예측이나 분류에 사용할 모델을 만드는 과정

데이터 분석 업무의 비중

- 데이터 분석 과정에서는 모델링보다 전처리에 많은 시간이 소요됨
- 모델링은 전처리를 마친 데이터를 바탕으로 진행



전처리의 중요성

* 집계(Aggregation) : 여러 데이터를 묶어 함께나 평균 같은 값으로 요약하는 작업

전처리의 중요성

- 데이터가 정확할수록 분석과 집계 과정의 오류가 줄어듦
- 잘못된 데이터를 입력하면 결과도 잘못될 수 있음
- 데이터 전처리 : 결측치 보정, 이상치 처리, 정규화, 인코딩을 수행하는 작업



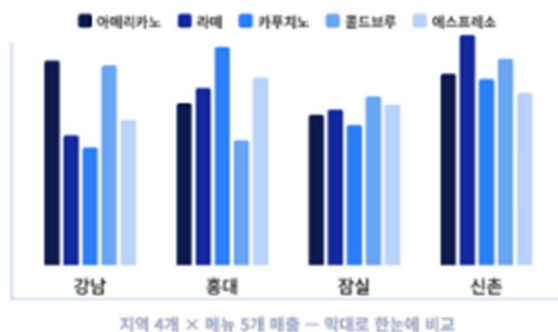
숫자표 vs 그림

메뉴 \ 지역	강남	홍대	잠실	신촌
아메리카노	12000	9500	8800	11200
라떼	7600	10400	9100	13500
카푸치노	6900	12800	8200	10900
콜드브루	11700	7300	9900	12100
에스프레소	8500	11000	9400	10100

지역 4개 × 메뉴 5개 매출 — 한눈에 패턴이 보이나요?

같은 데이터를 그림으로 보면

- 막대 길이를 이용하면 범주별 값의 차이를 비교하기 쉬움



차트를 선택하는 법

SSAFY

- 데이터 특성에 적합한 차트를 고르면 데이터 비교, 추이 분석, 분포 파악이 용이함
- 차트를 잘못 지정하면 데이터 정보를 왜곡하거나 흐리게 만들기 때문에 유의해야 함(시계열을 막대로 그리면 추이가 안 보임)

데이터 특성	추천 차트	용도	예시
범주별 크기 비교	막대그래프 bar	값 비교	지역별 매출
시간에 따른 변화	선그래프 line	추이 확인	월별 매출 변화
두 변수의 관계	산점도 scatter	관계 확인	단가와 수량
값의 분포와 빈도	히스토그램 hist	분포 확인	수량 분포
범주별 편차와 이상치	상자그림 boxplot	산포와 이상치 확인	지역별 매출 분포

* 편차(Deviation) : 각 값이 평균과 같은 기준값에서 떨어진 정도

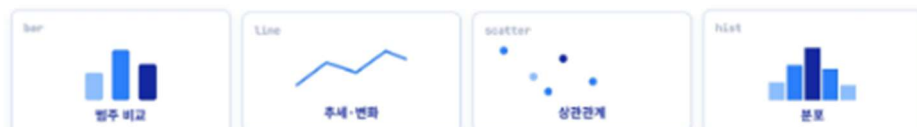
* 산포(Dispersion) : 데이터가 넓게 퍼져 있는 정도

* 시계열(Time Series) : 시간 순서에 따라 기록된 데이터

* 분포(Distribution) : 데이터 값이 어느 구간에 얼마나 나타나는지 보여주는 형태

기본 차트

- **bar** : 막대그래프로 범주별 값을 비교
- **line** : 꺾은선그래프로 시간에 따른 변화를 확인
- **scatter** : 산점도로 두 값의 관계를 확인
- **hist** : 히스토그램으로 값의 분포를 확인
- 값 비교 : 막대그래프
- 변화 추이 : 꺾은선그래프
- 변수 관계 : 산점도



한글 폰트 깨짐

※ Matplotlib : 파이썬에서 차트를 그릴 때 사용하는 시각화 라이브러리

한글 폰트 설정

- Matplotlib의 기본 폰트에서 한글이 네모로 표시될 수 있음
- Windows는 **Malgun Gothic**, macOS는 **AppleGothic**을 사용
- **axes.unicode_minus**를 **False**로 설정하면 음수 부호가 깨지는 현상을 방지할 수 있음

```
In [ ]: import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'Malgun Gothic' # Windows 한글 폰트
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 표시
```

폰트, 범례, 축 포매팅

※ 범례(legend) : 차트의 색상이나 선이 어떤 데이터를 나타내는지 설명하는 영역

폰트와 차트 구성

- 운영체제에 맞는 한글 폰트를 설정: Windows **Malgun Gothic**, macOS **AppleGothic**, Linux **NanumGothic**
- **plt.legend()** : 범례의 위치를 설정
- **plt.xticks(rotation=45)** : x축 글자를 45도 회전
- **plt.tight_layout()** : 차트 요소가 겹치지 않도록 여백을 조정

```
In [ ]: plt.figure(figsize=(10, 6)) # 크기 조정
plt.bar(x, y, label='매출')
plt.xlabel('지역', fontsize=12)
plt.ylabel('매출', fontsize=12)
plt.title('지역별 매출', fontsize=14)
plt.legend(loc='upper right')
plt.xticks(rotation=45)
plt.tight_layout() # 차트 요소가 겹치지 않도록 여백 조정
plt.show()
```

Seaborn 통계 시각화

Seaborn : Matplotlib을 기반으로 통계 차트를 만드는 라이브러리

- **barplot** : 범주별 평균을 비교
- **boxplot** : 값의 분포와 이상치를 확인
- **heatmap** : 상관관계와 같은 행렬 데이터를 표현
- 한글을 표시하려면 Matplotlib과 같은 방식으로 폰트를 설정

```
In [ ]: import seaborn as sns
```

```
sns.barplot(data=df, x='지역', y='매출') # 지역별 평균 매출  
sns.boxplot(data=df, x='매뉴', y='매출') # 매뉴별 매출 분포  
sns.heatmap(df.corr(numeric_only=True), annot=True) # 상관관계
```

* 상관관계(Correlation) : 한 변수가 변할 때 다른 변수가 함께 변하는 정도
* 행렬(Matrix) : 값을 행과 열로 배열한 2차원 형태의 데이터

전처리란

SSAFY

데이터 전처리

전처리 : 원본 데이터를 분석과 모델 학습에 맞게 다듬는 작업

- 데이터 불러오기, 전처리, 분석, 시각화, 모델 학습 순서로 진행
- 입력 데이터의 품질은 분석과 모델 결과에 영향을 줌
- 원본 데이터는 보존하고 전처리한 사본을 사용



전처리 4대 작업

전처리 주요 작업

- 결측치(Missing Value) : 비어 있는 값을 채우거나 해당 데이터를 제거
- 이상치(Outlier) : 다른 값보다 지나치게 크거나 작은 값을 확인하고 처리
- 정규화(Normalization) : 서로 다른 값의 범위를 같은 기준으로 조정
- 인코딩(Encoding) : 범주형 데이터를 숫자로 변환

작업	문제 상황	처리 방법
결측치	단가가 비어 있음	평균이나 중앙값으로 대체
이상치	수량이 99로 입력됨	값 확인 후 보정하거나 제거
정규화	단가와 수량의 범위가 다름	같은 범위로 변환
인코딩	매뉴가 문자열로 저장됨	숫자 형식으로 변환

결측치와 이상치 확인

- `df.isnull().sum()` : 컬럼별 결측치 개수를 확인
- `df.describe()` : 최솟값과 최댓값 등으로 이상치의 단서를 확인
- 상자그림과 히스토그램으로 값의 분포를 확인
- 값을 처리하기 전에 결측치와 이상치가 발생한 원인을 확인

```
df.isnull().sum()
```

```
age 3  
income 0  
city 5
```

```
df.describe()
```

```
age income  
mean 34 4200  
max 320 9900
```

quantile 분위수

분위수(quantile)

- `quantile()` : 데이터를 크기순으로 나눈 위치의 값을 계산
- `quantile(0.9)` : 하위 90%와 상위 10%를 나누는 경계값
- `0.25, 0.5, 0.75` : 데이터를 네 구간으로 나누는 사분위수
- `quantile(0.5)` : 중앙값
- 분위수는 이상치 판단과 상위 비율 필터에 사용

```
In [ ]: # 상위 10% 매출 경계값  
high = df['매출'].quantile(0.9)  
high  
  
# 상위 10% 주문만 필터  
df.loc[df['매출'] >= high]
```

차트 그리기

막대그래프와 꺾은선그래프

- `plt.bar(x, y)` : 막대그래프를 생성
- `plt.plot(x, y)` : 꺾은선그래프를 생성
- `title, xlabel, ylabel` : 제목과 축 이름을 설정
- `plt.show()` : 그래프를 출력

```
In [ ]: 지역별매출 = df.groupby('지역')['매출'].sum()  
  
plt.bar(지역별매출.index, 지역별매출.values)  
plt.title('지역별 매출')  
plt.xlabel('지역'); plt.ylabel('매출')  
plt.show()  
  
# 월별 매출 변화  
월별매출 = [120, 135, 128, 150, 162]  
plt.plot(range(1, 6), 월별매출, marker='o')  
plt.title('월별 매출 추이')  
plt.show()
```

서브플롯으로 여러 차트 배치

서브플롯(subplot)

- `plt.subplots(행, 열)`: 지정한 행과 열의 차트 영역을 생성
- 각 영역은 `axes[행, 열]`로 선택
- `plt.tight_layout()`: 차트가 겹치지 않도록 간격을 조정

```
In [ ]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))

지역별매출 = df.groupby('지역')['매출'].sum()
axes[0, 0].bar(지역별매출.index, 지역별매출.values)
axes[0, 0].set_title('지역별 매출')

axes[1, 0].hist(df['수량'], bins=5, edgecolor='black')
axes[1, 0].set_title('수량 분포')

axes[1, 1].scatter(df['단가'], df['수량'])
axes[1, 1].set_title('단가 vs 수량')

plt.tight_layout()
plt.show()
```

결측치 채우기

* NaN: 판다스에서 값이 비어 있는 결측치를 나타낼 때 주로 사용하는 표기

결측치 채우기

- `df["단가"].isnull().sum()`: 결측치 개수를 확인
- `fillna(값)`: 결측치를 지정한 값으로 대체
- 평균: 모든 값의 평균으로 결측치를 대체
- 중앙값: 이상치의 영향을 줄여 결측치를 대체
- 값을 대체하기 전에 선택한 값이 데이터에 적합한지 확인

```
In [ ]: print(df['단가'].isnull().sum())          # 결측치 개수
평균단가 = df['단가'].mean()                    # NaN을 제외하고 평균 계산
df['단가'] = df['단가'].fillna(평균단가)         # 평균으로 채움

# 중앙값으로 결측치 대체
df['단가'] = df['단가'].fillna(df['단가'].median())
```

- 평균은 이상치에 끌려가므로, 분포가 치우치면 중앙값(median)이 더 안전함
- `dropna()`로 결측 행을 통째로 버릴 수도 있으나 데이터 손실에 주의해야 함

결측치 대체, mode

최빈값으로 결측치 대체

- `mode()`: 가장 자주 나타나는 값인 최빈값을 반환
- 범주형 데이터의 결측치를 대체할 때 최빈값을 사용할 수 있음
- `mode()`는 Series를 반환하므로 첫 번째 값은 [0]으로 선택
- `notnull()`: 결측치가 아닌 행을 표시

```
In [ ]: # 최빈값으로 결측치 대체
df['지역'] = df['지역'].fillna(df['지역'].mode()[0])

# 단가가 있는 행의 매출 평균 계산
df.loc[df['단가'].notnull(), '매출'].mean()
```

* Series: 인덱스와 값이 짝을 이루는 판다스의 1차원 자료구조

Categorical 범주형 타입

범주형 자료형(category)

- `category` : 정해진 범주를 저장하는 자료형
- 반복되는 문자열을 범주형으로 변환하면 메모리 사용량을 줄일 수 있고, 범주의 순서도 지정할 수 있음
- 범주에 없는 값으로 결측치를 대체하려면 해당 값을 범주에 먼저 추가
- `.cat.add_categories()` : 새로운 범주를 추가 / `.cat.categories` : 현재 범주를 확인

```
In [ ]: # 지역을 category 타입으로 변환
df['지역'] = df['지역'].astype('category')

# 새 범주 추가 후 결측치 대체
df['지역'] = df['지역'].cat.add_categories('기타')
df['지역'] = df['지역'].fillna('기타')
```

drop 컬럼, 행 삭제

컬럼과 행 삭제

- `drop()` : 지정한 컬럼이나 행을 제거
- `axis=1` : 컬럼을 제거 / `axis=0` : 행을 제거
- 여러 항목을 제거할 때에는 목록으로 전달
- 원본을 유지하려면 결과를 변수에 저장

```
In [ ]:

# 분석에 필요 없는 컬럼 제거
df = df.drop(['메뉴'], axis=1)

df.head()
```

apply 함수 적용

apply() 함수 적용

- `apply()` : 컬럼의 각 값에 함수를 적용
- `def`로 정의한 함수를 값마다 실행해 새로운 값을 생성
- 조건식이 짧으면 `lambda`로 작성 가능
- 값에 따라 등급이나 코드를 지정할 때 사용

```
In [ ]: def grade(x):
    if x >= 5000:
        return '고가'
    else:
        return '저가'

df['가격대'] = df['단가'].apply(grade)
# lambda 로도 동일: df['단가'].apply(lambda x: '고가' if x >= 5000 else '저가')
```

※ 람다(Lambda) : 이름 없이 하나의 표현식으로 작성하는 익명 함수

qcut, cut 구간화

구간화

- 연속형 데이터를 여러 구간으로 나누어 범주형 데이터로 변환
- `pd.qcut(컬럼, n)` : 각 구간의 데이터 개수가 비슷하도록 n개 구간으로 나눔
- `pd.cut(컬럼, n)` : 값의 범위를 같은 폭의 n개 구간으로 나눔
- 결과는 구간을 나타내는 범주형 데이터이며, `value_counts()`로 구간별 데이터 개수를 확인

```
In [ ]: df['매출_구간'] = pd.qcut(df['매출'], 5) # 동일 빈도 5구간
        df['단가_구간'] = pd.cut(df['단가'], 3) # 동일 폭 3구간
        df['매출_구간'].value_counts()
```

* 연속형 데이터(Continuous Data) : 단가나 매출처럼 일정한 범위 안에서 연속된 값으로 측정되는 수치형 데이터

탐색적 데이터 분석 (EDA)

탐색적 자료 분석

SSAFY

탐색적 자료 분석 (Exploratory Data Analysis)

- 수집한 데이터가 들어왔을 때 이를 다양한 각도에서 관찰하고 이해하는 과정
 - 데이터를 분석하기 전에 그래프나 통계적인 방법으로 자료를 직관적으로 바라보는 과정
- 탐색적 자료 분석의 필요성
 - 데이터의 분포 및 값을 검토함으로써 데이터가 표현하는 현상을 이해
 - 데이터에 대한 잠재적인 문제를 발견, 본격적 분석 이전 데이터의 수집을 결정
 - 문제 정의 단계에서 미처 발견하지 못했을 다양한 패턴을 발견하고, 이를 바탕으로 기존의 가설을 수정하거나 새로운 가설을 세울 수 있음
- 탐색적 자료 분석은 데이터 과학에서 분석 과정의 핵심적인 부분으로 주요 과정은 다음과 같음
 - 분석의 목적 설계 및 데이터 확보
 - 데이터 탐색(Null, N/A, Outlier 등)
 - * Null, N/A : 값이 없거나 확인되지 않았음을 나타내는 결측 표기
 - 데이터 값 관찰 및 변환

탐색적 자료 분석 실습

STEP 1: 문제 정의 및 목표 설정

- 목표:
 - 타이타닉 호 승객데이터를 기반으로 생존에 영향을 미치는 요인 분석
- 주요 과제:
 - 기본 EDA (Exploratory Data Analysis)
 - 데이터 전처리 (불필요한 데이터 삭제, 추가, 변경)
 - 인사이트 발굴

STEP 2: 모듈 import

```
In [ ]: from IPython.display import Image
import numpy as np
import pandas as pd
import seaborn as sns
```

※ 모듈(Module) : import로 불러와 사용하는 파이썬 코드 파일
※ 판다스(Pandas) : 표 형태의 데이터를 처리하는 파이썬 라이브러리
※ 넘파이(Numpy) : 수치 계산과 배열 연산에 사용하는 파이썬 라이브러리
※ 인사이트(Insight) : 데이터를 분석해 확인한 의미 있는 정보나 결론

탐색적 자료 분석 실습

SSAFY

STEP 3: 데이터셋 로드

```
In [ ]: df = sns.load_dataset('titanic')
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

탐색적 자료 분석 실습

데이터 컬럼 설명

- survived: 생존여부 (1: 생존, 0: 사망)
- pclass: 좌석 등급 (1등급, 2등급, 3등급)
- sex: 성별
- age: 나이
- sibsp: 형제 + 배우자 수
- parch: 부모 + 자녀 수
- fare: 좌석 요금
- embarked: 탑승 항구 (S, C, Q)
- class: pclass와 같은 정보를 문자열로 표현 (First, Second, Third)
- who: 성인 남성/성인 여성/아동 구분 (man, woman, child)
- adult_male: 성인 남자 여부
- deck: 객실 데크 (A~G, 결측 688건)
- embark_town: 탑승 항구 이름
- alive: 생존여부 (yes, no)
- alone: 혼자 탑승 여부

탐색적 자료 분석 실습

STEP 4: 기본 데이터 조회

- 상위 5개의 행 출력

```
In [ ]: # 코드를 입력해 주세요
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	25.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	5.5000	S	Third	man	True	NaN	Southampton	no	True

- 하위 5개의 행 출력

```
In [ ]: # 코드를 입력해 주세요
df.tail()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
886	0	2	male	27.0	0	0	13.00	S	Second	man	True	NaN	Southampton	no	True
887	1	1	female	19.0	0	0	30.00	S	First	woman	False	B	Southampton	yes	True
888	0	3	female	NaN	1	2	23.45	S	Third	woman	False	NaN	Southampton	no	False
889	1	1	male	25.0	0	0	30.00	C	First	man	True	C	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.75	G	Third	man	True	NaN	Queenstown	no	True

탐색적 자료 분석 실습

* dtype : 배열이나 컬럼에 저장된 값의 자료형

STEP 4: 기본 데이터 조회

- 데이터가 몇 개의 행과 열로 이루어져 있는지 확인

```
In [ ]: # 코드를 입력해 주세요
df.shape
```

```
Out[ ]: (891, 15)
```

- 컬럼별 데이터의 dtype과 개수를 확인

```
In [ ]: # 코드를 입력해 주세요
df.info()
```

```
Out[ ]: <class 'pandas.core.frame.DataFrame'>
Int64Index: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column             Non-Null Count  Dtype  
---  -
0   survived           891 non-null    int64  
1   pclass             891 non-null    int64  
2   sex                891 non-null    object  
3   age                714 non-null    float64
4   sibsp              891 non-null    int64  
5   parch              891 non-null    int64  
6   fare               891 non-null    float64
7   embarked           891 non-null    object  
8   embarked_c         891 non-null    category
9   who                891 non-null    object  
10  adult_male         891 non-null    bool    
11  deck               205 non-null    category
12  embark_town        891 non-null    object  
13  alive              891 non-null    bool    
14  alone              891 non-null    bool    
dtypes: bool(3), category(2), float64(1), int64(4), object(5)
memory usage: 83.3+ KB
```

탐색적 자료 분석 실습

STEP 4: 기본 데이터 조회

- 데이터의 컬럼별 결측치 확인

```
In [ ]: # 코드를 입력해 주세요
df.isnull().sum()
```

```
Out[ ]: survived      0
        dt class      0
        sex          0
        age         177
        sibsp        0
        parch        0
        fare         0
        embarked     2
        class        0
        who          0
        adult_male    0
        deck        988
        embark_town   2
        alive         0
        dtype: int64
```

- 생존자와 사망자의 분포 확인

```
In [ ]: # 코드를 입력해 주세요
df['survived'].value_counts()
```

```
Out[ ]: 0    549
        1    942
        Name: survived, dtype: int64
```

STEP 5: 탐색적 데이터 분석 (EDA)

- 항구별 생존자 합계 및 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('embarked')['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
embarked
C      93  0.553571
Q      30  0.389610
S     217  0.336957
```

- 성별 생존자 합계 및 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('sex')['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
sex
female  233  0.742038
male    109  0.186908
```

STEP 5: 탐색적 데이터 분석 (EDA)

- 성별, 등급별 생존자 합계 & 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby(['sex', 'pclass'])['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
sex pclass
female 1  91  0.306586
       2  71  0.421132
       3  72  0.305556
male    1  40  0.350000
       2  17  0.117647
       3  47  0.125447
```

- 혼자인 경우 / 성별 합계 & 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby(['alone', 'sex'])['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
alone sex
False female  154  0.712708
       male    45  0.271504
True  female   99  0.736214
       male    64  0.156719
```

* groupby: 특정 컬럼을 기준으로 행을 그룹으로 묶어 집계하는 기능

STEP 5: 탐색적 데이터 분석 (EDA)

- 항구별 생존자 합계를 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('embarked')['survived'].sum()
```

```
Out[ ]: embarked
C      93
Q      30
S     217
Name: survived, dtype: int64
```

- 항구별 생존율을 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('embarked')['survived'].mean()
```

```
Out[ ]: embarked
C      0.553571
Q      0.389610
S      0.336957
Name: survived, dtype: float64
```

STEP 5: 탐색적 데이터 분석 (EDA)

- 혼자인 경우와 혼자가 아닌 경우 합계 & 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('alone')['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
alone
False  179  0.509050
True   163  0.303538
```

- 등급(pclass)별 생존자 합계 & 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('pclass')['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
pclass
1      136  0.629630
2       87  0.472826
3      119  0.242363
```

STEP 5: 탐색적 데이터 분석 (EDA)

- who, 등급별 생존자 합계 & 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby(['who', 'pclass'])['survived'].agg(['sum', 'mean'])
```

```
Out[ ]:      sum  mean
who  pclass
child 1     5  0.833333
      2    19  1.000000
      3    25  0.431034
man    1    42  0.352941
      2     8  0.080808
      3    38  0.119122
woman 1    89  0.978022
      2    60  0.909091
      3    56  0.491228
```

STEP 5: 탐색적 데이터 분석 (EDA)

- pivot_table 을 활용하여, (sex, pclass) 별 생존자 합계(sum) 출력

```
In [ ]: df.pivot_table(index='sex',
                        columns='pclass',
                        values='survived',
                        aggfunc=['sum', 'mean'])
```

```
Out[ ]:      sum      mean
pclass sex 1 2 3 1 2 3
female 91 70 72 0.968085 0.921053 0.505000
male   45 17 47 0.368852 0.157407 0.135447
```

pivot_table : 행과 열의 기준에 따라 값을 교차 집계한 표

STEP 5: 탐색적 데이터 분석 (EDA)

- who, 등급별 생존자 합계 & 생존율 계산 결과를 토대로 별도의 DataFrame으로 생성
- 인덱스 초기화 및 생존율 내림차순 정렬

```
In [ ]: # 코드를 입력해 주세요
result = df.groupby(['who', 'pclass'])['survived'].agg(['sum', 'mean'])
result = result.reset_index()
result.sort_values(by='mean', ascending=False).reset_index(drop=True)
```

```
Out[ ]:      who  pclass  sum  mean
0  child      2    19  1.000000
1  woman      1    89  0.978022
2  woman      2    60  0.909091
3  child      1     5  0.833333
4  woman      3    56  0.491228
5  child      3    25  0.431034
6  man        1    42  0.352941
7  man        3    38  0.119122
8  man        2     8  0.080808
```

데이터프레임(DataFrame) : 행과 열로 구성된 표 형태의 판다스 자료구조
인덱스(index) : 데이터프레임의 각 행을 구분하는 라벨

STEP 5: 탐색적 데이터 분석 (EDA)

- child의 age는 몇 세부터 몇 세까지로 정의되어 있는지 확인(hint: .loc와 agg를 활용)

```
In [ ]: # 코드를 입력해 주세요
df.loc[df['who'] == 'child', 'age'].agg(['min', 'max'])
```

```
Out[ ]: min    0.42
        max   15.00
        Name: age, dtype: float64
```

- 등급별(pclass) / 연령별(who) 평균 요금 비교

```
In [ ]: # 코드를 입력해 주세요
pd.DataFrame(df.groupby(['pclass', 'who'])['fare'].mean())
```

```
Out[ ]:
```

		fare
	child	130.342033
1	man	65.951906
	woman	104.317985
	child	25.323935
2	man	19.254124
	woman	30.86924
	child	25.221990
3	man	16.340212
	woman	15.2341201

STEP 5: 탐색적 데이터 분석 (EDA)

- 부자는 살았을까? (fare 요금 기준 상위 10%의 생존율 확인)
 - fare 기준 상위 10% 기준 금액 확인 및 생존자 & 생존율 계산

```
In [ ]: # 코드를 입력해 주세요
rich = df['fare'].quantile(0.9)
rich
```

```
Out[ ]: 77.9583
```

```
In [ ]: # 코드를 입력해 주세요
df.loc[df['fare'] >= rich, 'survived'].agg(['count', 'mean'])
```

```
Out[ ]: count    90.000000
        mean      0.766667
        Name: survived, dtype: float64
```

STEP 5: 탐색적 데이터 분석 (EDA)

- 생존자의 평균 나이와 사망자의 평균 나이 비교

```
In [ ]: # 코드를 입력해 주세요
df.groupby('survived')['age'].mean()
```

```
Out[ ]: survived
0      30.626179
1      28.343690
        Name: age, dtype: float64
```

- deck 정보가 NaN인 경우와 채워져 있는 경우 생존율 비교

```
In [ ]: # 코드를 입력해 주세요
df.loc[df['deck'].isnull(), 'survived'].mean()
```

```
Out[ ]: 0.23941850465116277
```

```
In [ ]: # 코드를 입력해 주세요
df.loc[df['deck'].notnull(), 'survived'].mean()
```

```
Out[ ]: 0.6699507389162561
```

STEP 6: 전처리 (pre-processing)

- 결측치 대체

- embarked 컬럼은 최빈값(mode)으로 대체

```
In [ ]: # 코드를 입력해 주세요
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
```

```
In [ ]: # 코드 검증 (Cell 실행시 오류가 나지 않아야 함)
assert 0 == df['embarked'].isnull().sum()
```

STEP 6: 전처리 (pre-processing)

- 결측치 확인

```
In [ ]: # 코드를 입력해 주세요
df.isnull().sum()
```

```
Out[ ]: survived    0
        pclass      0
        sex         0
        age         177
        sibsp       0
        parch       0
        fare        0
        embarked    2
        class       0
        who         0
        adult_male  0
        deck        600
        embark_town 2
        alive       0
        alone       0
        dtype: int64
```

STEP 6: 전처리 (pre-processing)

• 결측치 대체

- age 컬럼은 sex에 따른 평균 age로 대체. 남자 -> 평균 age, 여자 -> 평균 age
- 평균 age 계산

```
In [ ]: # 코드를 입력해 주세요
df.groupby('sex')['age'].mean()
```

```
Out[ ]: Sex
female    27.915709
male      30.726645
Name: age, dtype: float64
```

STEP 6: 전처리 (pre-processing)

• 결측치 대체

```
In [ ]: # 코드를 입력해 주세요
male_mean = df.groupby('sex')['age'].mean()['male']
female_mean = df.groupby('sex')['age'].mean()['female']

df.loc[df['sex'] == 'male', 'age'] = \
    df.loc[df['sex'] == 'male', 'age'].fillna(male_mean)
df.loc[df['sex'] == 'female', 'age'] = \
    df.loc[df['sex'] == 'female', 'age'].fillna(female_mean)
```

```
In [ ]: # 코드 검증 (Cell 실행시 예러가 나지 않아야 함)
assert 27.92 == round(df.groupby('sex')['age'].mean()['female'], 2)
assert 30.73 == round(df.groupby('sex')['age'].mean()['male'], 2)
```

STEP 6: 전처리 (pre-processing)

• 결측치 대체

- deck 컬럼 결측치는 No Data로 대체

```
In [ ]: # 코드를 입력해 주세요
df['deck'] = df['deck'].cat.add_categories('No Data')
```

```
In [ ]: # 코드를 입력해 주세요
df['deck'] = df['deck'].fillna('No Data')
```

```
In [ ]: df['deck'].value_counts()
```

```
Out[ ]: No Data    688
C                59
B                47
D                33
E                32
A                15
F                13
G                 4
Name: deck, dtype: int64
```

STEP 6: 전처리 (pre-processing)

• 중복된 컬럼 제거 (class, embark_town, alive 컬럼 제거)

```
In [ ]: df.head()
```

	survived	pclass	sex	age	sibsp	parth	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	No Data	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.8250	S	Third	woman	False	No Data	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	No Data	Southampton	no	True

```
In [ ]: # 코드를 입력해 주세요
df = df.drop(['class', 'embark_town', 'alive'], axis=1)
```

```
In [ ]: df.head()
```

	survived	pclass	sex	age	sibsp	parth	fare	embarked	who	adult_male	deck	alone
0	0	3	male	22.0	1	0	7.2500	S	man	True	No Data	False
1	1	1	female	38.0	1	0	71.2833	C	woman	False	C	False
2	1	3	female	26.0	0	0	7.8250	S	woman	False	No Data	True
3	1	1	female	35.0	1	0	53.1000	S	woman	False	C	False
4	0	3	male	35.0	0	0	8.0500	S	man	True	No Data	True

STEP 6: 전처리 (pre-processing)

• 특성 공학 (feature engineering)

- family 컬럼을 만들고 sibsp + parch 더한 값을 입력

```
In [ ]: # 코드를 입력해 주세요
df['family'] = df['sibsp'] + df['parch']
```

```
In [ ]: df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	who	adult_male	deck	alone	family
0	0	3	male	22.0	1	0	7.2500	S	man	True	No Data	False	1
1	1	1	female	38.0	1	0	71.2833	C	woman	False	C	False	1
2	1	3	female	26.0	0	0	7.9250	S	woman	False	No Data	True	0
3	1	1	female	35.0	1	0	53.1000	S	woman	False	C	False	1
4	0	3	male	35.0	0	0	8.0500	S	man	True	No Data	True	0

※ 특성 공학 (Feature Engineering) : 기존 데이터를 조합하거나 변환해 분석이나 모델링에 사용할 새로운 컬럼을 만드는 작업

STEP 6: 전처리 (pre-processing)

• sex&family 별 생존을 확인

```
In [ ]: # 코드를 입력해 주세요
pd.DataFrame(df.groupby(['sex', 'family'])['survived'].mean())
```

```
Out[ ]:
survived
sex family
0 0.785714
1 0.816667
2 0.775510
3 0.842105
female 4 0.250000
5 0.375000
6 0.375000
7 0.000000
10 0.000000
8 0.155718
9 0.243243
3 0.960304
2 0.500000
male 4 0.000000
5 0.000000
6 0.250000
7 0.000000
10 0.000000
```

STEP 6: 전처리 (pre-processing)

• 생존을 TOP 5 출력

```
In [ ]: # 코드를 입력해 주세요
result = df.groupby(['sex', 'family'])['survived'].mean()
result = result.reset_index().sort_values('survived', ascending=False)
result.head().reset_index(drop=True)
```

```
Out[ ]:
sex family survived
0 female 3 0.842105
1 female 1 0.816667
2 female 0 0.785714
3 female 2 0.775510
4 male 3 0.500000
```

STEP 6: 전처리 (pre-processing)

• 생존을 하위 TOP 10 출력

```
In [ ]: # 코드를 입력해 주세요
result = df.groupby(['sex', 'family'])['survived'].mean()
result = result.reset_index().sort_values('survived', ascending=False)
result.tail(10).reset_index(drop=True)
```

```
Out[ ]:
sex family survived
0 female 4 0.250000
1 male 6 0.250000
2 male 1 0.243243
3 male 0 0.155718
4 female 10 0.000000
5 male 4 0.000000
6 male 5 0.000000
7 female 7 0.000000
8 male 7 0.000000
9 male 10 0.000000
```


STEP 6: 전처리 (pre-processing)

- apply 함수를 활용하여 남자는 1, 여자는 0으로 값을 변경하고, gender 컬럼을 새로 만들어 적용

```
In [ ]: # 코드를 입력해 주세요
def make_bin(x):
    if x == 'male':
        return 1
    elif x == 'female':
        return 0
    return 0

df['gender'] = df['sex'].apply(make_bin)

In [ ]: # 코드 검증
df['gender'].value_counts()

Out[ ]: 1    577
        0    314
        Name: gender, dtype: int64
```

STEP 6: 전처리 (pre-processing)

- 나이를 10구간으로 나누어 age_bin 컬럼을 새로 만들어 적용
- 동일한 구간을 갖도록 pd.cut()을 사용

```
In [ ]: # 코드를 입력해 주세요
df['age_bin'] = pd.cut(df['age'], 10)

In [ ]: df['age_bin'].value_counts()

Out[ ]: (24.294, 32.252]    346
        (16.336, 24.294]    177
        (32.252, 40.21]    118
        (40.21, 48.168]    70
        (0.34, 8.376]      54
        (8.376, 16.336]    46
        (48.168, 56.126]    45
        (56.126, 64.084]    24
        (64.084, 72.042]    9
        (72.042, 80.0]      2
        Name: age_bin, dtype: int64
```

STEP 6: 전처리 (pre-processing)

- 요금을 5구간으로 나누어 fare_bin 컬럼을 새로 만들어 적용
- 동일한 분포를 갖도록 pd.qcut()을 사용

```
In [ ]: # 코드를 입력해 주세요
df['fare_bin'] = pd.qcut(df['fare'], 5)

In [ ]: df['fare_bin'].value_counts()

Out[ ]: (7.854, 10.5]      184
        (21.679, 39.688]   180
        (-9.001, 7.854]    179
        (39.688, 512.329]  176
        (10.5, 21.679]    172
        Name: fare_bin, dtype: int64
```

핵심 키워드

주제	키워드	핵심
Matplotlib 기본 차트	bar(), plot(), scatter(), hist(), 한글 폰트	값 비교, 변화 추이, 변수 관계, 분포를 차트로 표현
Seaborn 통계 시각화	barplot(), boxplot(), heatmap()	범주별 평균, 값의 분포, 상관관계를 시각화
데이터 전처리	fillna(), mode(), quantile()	결측치와 이상치를 확인하고 평균, 중앙값, 최빈값, 분위수로 처리
데이터 변환	apply(), cut(), qcut(), category	함수 적용, 구간화, 범주형 자료형 변환
탐색적 데이터 분석 (EDA)	describe(), groupby(), pivot_table(), value_counts()	데이터를 요약하고 집계해 특징과 패턴을 확인

오늘 공부한 내용 요약 및 정리

- 막대그래프: 범주별 값을 비교
- 꺾은선그래프: 시간에 따른 값의 변화를 확인
- 산점도: 두 변수의 관계를 확인
- 히스토그램: 값의 분포를 확인
- 한글이 깨지면 한글 폰트를 지정
- 전처리: 분석과 모델 학습 전에 결측치와 이상치를 처리
- 범주형 데이터: 필요한 경우 숫자 형식으로 인코딩
- describe(): 수치형 데이터의 요약 통계를 확인
- value_counts(): 값별 개수를 확인
- 상자그림과 히스토그램: 이상치와 분포를 확인